

SubTrack

TECHNICAL DOCUMENTATION

Version 1.0

Platform: iOS · Android · Web (Expo SDK 53)

Primary market: Kingdom of Saudi Arabia

Languages: English · Arabic (RTL)

July 2026

SubTrack — Technical Documentation

Table of Contents

1. [Overview](#overview)
 2. [Architecture](#architecture)
 3. [Authentication](#authentication)
 4. [Data Models](#data-models)
 5. [Screens Reference](#screens-reference)
 6. [Feature: Dashboard](#feature-dashboard)
 7. [Feature: Subscriptions](#feature-subscriptions)
 8. [Feature: Bills](#feature-bills)
 9. [Feature: Loans](#feature-loans)
 10. [Feature: Savings Goals](#feature-savings-goals)
 11. [Feature: Budgets](#feature-budgets)
 12. [Feature: Bank Connections](#feature-bank-connections)
 13. [Feature: Analytics](#feature-analytics)
 14. [Feature: AI Advisor](#feature-ai-advisor)
 15. [Feature: Premium](#feature-premium)
 16. [Feature: Settings](#feature-settings)
 17. [API Reference](#api-reference)
 18. [Localization](#localization)
-

Overview

SubTrack is a personal finance management mobile application purpose-built for the Saudi Arabian market. It gives users a single place to track subscriptions, recurring bills, BNPL loans, and savings goals — with optional bank connectivity via the Lean Technologies open-banking API.

Property	Value
Platform	iOS · Android · Web (Expo SDK 53)
Primary market	Kingdom of Saudi Arabia
Currency	SAR (Saudi Riyal)
Languages	English · Arabic (RTL)
Backend	Node.js / Fastify + PostgreSQL
AI	OpenAI GPT-4o (streaming SSE)
Open banking	Lean Technologies
Monetization	Freemium — SAR 19.99 / month

Architecture

Monorepo Layout

SubTrack is a pnpm workspace monorepo. Each directory under `artifacts/` is an independent package with its own `package.json`.

```
/
% % % artifacts/
%   % % % mobile/           # Expo React Native app (@workspace/mobile)
%   % % % api-server/       # Fastify REST API + AI proxy (@workspace/api-server)
%   % % % subtrack-deck/    # Pitch deck slides artifact
%   % % % mockup-sandbox/   # Component preview server (dev only)
% % % packages/
%   % % % db/               # Shared Drizzle ORM schema + migrations (@workspace/db)
% % % docs/                 # This documentation
% % % pnpm-workspace.yaml
```

Mobile App (`artifacts/mobile/`)

Framework: Expo SDK 53 · React Native · Expo Router (file-based routing)

Directory Structure

```
artifacts/mobile/
% % % app/
%   % % % (tabs)/           # Bottom-tab screens (index, subscriptions, banks, analytics,
%   % % % settings, ...)    # Login, register screens
%   % % % auth/             # [id].tsx – subscription detail
%   % % % subscription/     # AI advisor chat
%   % % % agent.tsx         # Category budget caps
%   % % % budgets.tsx       # Lean bank-connection wizard
%   % % % lean-setup.tsx    # Premium upgrade screen
%   % % % premium.tsx      # Shared UI components
% % % components/          # React context providers (state management)
% % % context/             # Custom hooks
% % % hooks/               # en.ts · ar.ts translation strings
% % % locales/            # Icons, splash, fonts
% % % assets/
```

State Management

All app state is managed via React Context + AsyncStorage (local-first). There is no Redux or Zustand.

Context	Responsibility
`AuthContext`	User profile, login/logout, Lean tokens, income, premium status
`SubscriptionContext`	Subscriptions list, connected banks, `monthlyTotal`
`BillsContext`	Recurring bills
`LoanContext`	Credit cards, BNPL, personal loans
`GoalsContext`	Savings goals and contribution history
`BudgetContext`	Per-category spending caps
`LanguageContext`	Active language (`en`/`ar`), `t()` translation function, `isRTL` flag
`ThemeContext`	Theme override (`system`/`light`/`dark`), `resolvedScheme`

Data Persistence

- **Local:** All financial data (subscriptions, bills, loans, goals, budgets) is stored in AsyncStorage on-device. No network request is required to use the app.
- **Cloud sync:** When a user registers and logs in, their profile (name, email, premium status) is synced with the API server. Financial data itself is not synced to the cloud in the current version.

API Server (`artifacts/api-server/`)

Framework: Node.js · Fastify · Drizzle ORM · PostgreSQL

```
artifacts/api-server/src/
% % % routes/
%   % % % auth.ts      # Register, login, profile
%   % % % premium.ts   # Premium entitlement check / grant / revoke
%   % % % agent.ts      # AI advisor SSE streaming endpoint
%   % % % health.ts     # Health check
% % % middlewares/
%   % % % jwtAuth.ts    # Bearer token validation
% % % index.ts          # Server entry point
```

External Services

Service	Purpose
OpenAI GPT-4o	AI advisor responses (via Replit AI Integrations proxy)
Lean Technologies	Open-banking: bank account connection, income verification
PostgreSQL	User accounts and premium entitlements

Database (`packages/db/`)

Shared schema package used by the API server via Drizzle ORM.

Tables

Table	Purpose
`server_users`	Identity: email (PK), password hash, display name, device user ID
`premium_entitlements`	Premium status: email (PK), `is_premium`, `granted_at`, `updated_at`

Request Flow

```
Mobile App (Expo)
  %
  %   HTTPS (REST + SSE)
  %
API Server (Fastify)      !•% % % %   JWT middleware on protected routes
  %
  % % % % %   PostgreSQL (Drizzle ORM)
  %
  % % % % %   OpenAI API (AI advisor)  !•% %   Replit AI Integrations proxy
```

Environment Variables

Variable	Where set	Purpose
`EXPO_PUBLIC_API_URL`	Mobile (development + production)	Base URL for API calls — ends in `/api`
`SESSION_SECRET`	API server	JWT signing secret
`AI_INTEGRATIONS_OPENAI_API_KEY`	API server	OpenAI proxy key
`AI_INTEGRATIONS_OPENAI_BASE_URL`	API server	OpenAI proxy base URL
`DATABASE_URL`	API server	PostgreSQL connection string

Authentication

SubTrack is local-first — the app works fully without an account. Authentication is optional and unlocks cloud-backed premium status and cross-device profile restoration.

Auth Flow

Guest (no account)

All financial data (subscriptions, bills, loans, goals, budgets) is stored in AsyncStorage on the device. No login is required.

Registration

1. User navigates to **'Settings ! Sign up'**.
2. Provides name, email, and password.
3. App sends `POST /api/auth/register` with `{ email, passwordHash, name, userId }`.
 - `passwordHash` is a client-side hash of the password — the raw password never leaves the device.
 - `userId` is the locally generated UUID already in `AuthContext`, so data created before registration is associated with the account.
4. On success, the app stores the JWT returned by the subsequent login call.

Login

1. User provides email and password.
2. App sends `POST /api/auth/login` with `{ email, passwordHash }`.
3. Server validates credentials against `server_users.password_hash`.
4. Returns a **JWT** (90-day TTL, signed with `SESSION_SECRET`).
5. JWT is stored locally and attached as `Authorization: Bearer <token>` on all subsequent authenticated requests.

Profile Restoration

On app launch, if a JWT is present, the app calls `GET /api/auth/profile` to restore the user's name and email. This covers reinstalls and new devices.

Premium Status Sync

After login, the app calls `GET /api/premium/:email` to check premium entitlement. The result is stored in `AuthContext.user.isPremium` and persisted in AsyncStorage.

JWT Details

Property	Value
Algorithm	HS256
TTL	90 days
Claim	`sub` = user email
Header	`Authorization: Bearer <token>`

The JWT middleware (`middlewares/jwtAuth.ts`) decodes the token and sets `req.authEmail` on the Fastify request object. Routes that use `jwtAuth` will return `401` if the token is missing or invalid.

Auth Screens

Screen	Route
Auth landing	`/auth`
Login	`/auth/login`
Register	`/auth/register`

Security Notes

- Passwords are hashed **on the client** before transmission. The server stores only the hash.
 - JWTs are never sent to Lean Technologies — Lean uses its own `leanAppToken` stored separately in `AuthContext`.
 - Premium grant/revoke endpoints (`POST /api/premium/:email`, `DELETE /api/premium/:email`) are **disabled in production** and return `403`.
-

Data Models

All entities are stored locally in AsyncStorage. The TypeScript interfaces below are the canonical shapes used throughout the app.

UserProfile

```
interface UserProfile {
  id: string; // Local UUID, generated on registration
  name: string;
  email: string;
  passwordHash?: string; // Stored locally; not sent to server after auth
  isPremium?: boolean;
  income?: number; // Monthly net income in SAR
  leanVerifiedAt?: string; // ISO timestamp – set when income came from Lean open-
  banking
  leanAppToken?: string; // Lean Technologies app token
  leanCustomerId?: string; // Lean customer_id (returned after customer creation)
  kycCompleted?: boolean; // True after first successful Lean KYC flow
  createdAt: string; // ISO timestamp
}
```

Subscription

```
interface Subscription {
  id: string; // UUID
  name: string; // Service name (e.g. "Netflix")
  merchantName?: string; // Billing statement merchant name
  category: SubscriptionCategory;
  amount: number; // Amount in SAR (base price before VAT)
  billingCycle: 'weekly' | 'monthly' | 'annual';
  startDate: string; // ISO date string
  status: 'active' | 'cancelled';
  includeVat?: boolean; // Adds 15% VAT to displayed amount
  isShared?: boolean; // Whether cost is split with others
  sharedMembers?: number; // Number of people sharing (1-6)
  isTrial?: boolean;
  trialEndDate?: string; // DD/MM/YYYY
  bankAccountId?: string; // Linked bank account ID (from Lean)
  notes?: string;
}

type SubscriptionCategory =
  | 'Streaming'
  | 'Software'
  | 'Fitness'
  | 'Food'
  | 'Gaming'
```

Derived values (computed, not stored):

- ``shareCost = amount / sharedMembers`` — the user's personal share
- ``monthlyAmount`` — normalised to monthly: $\text{weekly} \times 4.33$, $\text{annual} \div 12$
- ``yearlyEquivalent`` — $\text{monthly equivalent} \times 12$

Bill

```
interface Bill {
  id: string;
  name: string;
  amount: number;           // SAR
  dueDay: number;           // Day of month (1-31)
  category: BillCategory;
  isPaid?: boolean;         // Toggled per cycle
}

type BillCategory =
  | 'Rent'
  | 'Insurance'
  | 'Utilities'
  | 'Phone'
  | 'Internet'
  | 'Other';
```

Loan

```
interface Loan {
  id: string;
  name: string;              // Descriptive label (e.g. "Tamara - iPhone")
  lender?: string;
  type: 'credit_card' | 'bnpl' | 'personal' | 'other';

  // Balance & limits
  currentBalance: number;    // SAR — remaining balance
  originalAmount?: number;   // SAR — original loan amount
  creditLimit?: number;      // SAR — for credit cards

  // Repayment
  apr?: number;              // Annual Percentage Rate (%)
  minimumPayment?: number;   // SAR — monthly minimum
  nextDueDate?: string;      // ISO date

  // BNPL-specific
  installmentAmount?: number; // SAR per installment
  totalInstallments?: number;
  paidInstallments?: number;
}
```

SavingsGoal

```
interface SavingsGoal {
  id: string;
  emoji: string;           // Visual identifier (e.g. "🏠", "👤")
  title: string;
  targetAmount: number;    // SAR
  currentAmount: number;   // SAR – accumulated savings
  deadline?: string;       // DD/MM/YYYY Gregorian
  color?: string;          // Hex accent colour for progress bar
  createdAt: string;
}
```

Derived values:

- ``progress`` = `currentAmount / targetAmount` (0–1)
- ``monthlyRateNeeded`` — amount needed per month to hit deadline
- ``isAtRisk`` — true when ``monthlyRateNeeded`` > 20% of user income

BudgetCap

```
interface BudgetCap {
  category: SubscriptionCategory;
  limitAmount: number;    // SAR per month
}
```

Budget health thresholds:

- **Normal** — spending < 80% of cap
- **Near limit** — spending 80–100% of cap
- **Over limit** — spending > cap

BankAccount

```
interface BankAccount {
  id: string;              // Local UUID
  bankName: string;
  accountType: string;     // e.g. "Current", "Savings"
  lastFour: string;        // Last 4 digits of account number
  leanEntityId?: string;   // Lean entity ID for API calls
  balanceAvailable?: number; // SAR – fetched from Lean
  iban?: string;
}
```

Server-side (PostgreSQL)

server_users

Column	Type	Notes
`email`	text (PK)	Primary key
`password_hash`	text	Bcrypt hash
`name`	text	Display name
`user_id`	text	Device-generated UUID
`created_at`	timestamp	

premium_entitlements

Column	Type	Notes
`email`	text (PK)	Foreign key to server_users
`is_premium`	boolean	
`granted_at`	timestamp	
`updated_at`	timestamp	

Screens Reference

All screens use Expo Router file-based routing. The bottom tab bar is always visible on the main screens.

Tab Screens

File	Route	Tab label
`app/(tabs)/index.tsx`	`/`	Dashboard
`app/(tabs)/subscriptions.tsx`	`/subscriptions`	Subscriptions
`app/(tabs)/bills.tsx`	`/bills`	Bills
`app/(tabs)/loans.tsx`	`/loans`	Loans
`app/(tabs)/analytics.tsx`	`/analytics`	Analytics
`app/(tabs)/banks.tsx`	`/banks`	Banks
`app/(tabs)/goals.tsx`	`/goals`	Goals
`app/(tabs)/settings.tsx`	`/settings`	Settings

Stack Screens (pushed modally or via link)

File	Route	Description
`app/subscription/[id].tsx`	`/subscription/:id`	Full detail view for a single subscription
`app/agent.tsx`	`/agent`	AI financial advisor chat
`app/budgets.tsx`	`/budgets`	Category-by-category monthly budget caps
`app/premium.tsx`	`/premium`	Premium upgrade / downgrade screen
`app/lean-setup.tsx`	`/lean-setup`	Lean Technologies API token setup

Auth Screens

File	Route	Description
`app/auth/index.tsx`	`/auth`	Auth landing (login / register choice)
`app/auth/login.tsx`	`/auth/login`	Login form

<code>`app/auth/register.tsx`</code>	<code>`/auth/register`</code>	Registration form
--------------------------------------	-------------------------------	-------------------

Navigation Guards

- **Subscriptions screen** — requires at least one connected bank account before the list is shown; otherwise displays a "Connect a bank" gate.
 - **Budgets screen** — wrapped in `<PremiumGate>`; non-premium users see an upgrade prompt instead of the budget list.
 - **Bills screen** — wrapped in `<PremiumGate>`.
 - **AI Advisor** — wrapped in `<PremiumGate>`.
 - **Loans screen** — wrapped in `<PremiumGate>`.
-

Feature: Dashboard

Route: `/` · File: `app/(tabs)/index.tsx`

The Dashboard is the first screen the user sees after launch. It provides a single-glance summary of the user's complete financial picture.

Summary Cards

Card	Description
Total Monthly Outflow	Sum of all active subscriptions (at share cost), bills, and loan minimum payments for the current month
Subscriptions	Active subscription count and their combined monthly cost
Bills	Total monthly bill obligations
Loans	Total minimum monthly loan payments and aggregate outstanding debt

Top Spending Category

Shows the single highest-spend category across subscriptions, bills, and loans — displayed with a colour-coded icon and the monthly amount attributed to it.

Upcoming Payments

A chronological list of the next payments due within the current month, including:

- Subscription renewal dates (derived from `startDate` + billing cycle)
- Bill due days
- Loan next-due dates

Each row shows the service name, amount, and days remaining.

Savings Rate Indicator

When the user has set their monthly income (manually or via Lean), the dashboard shows a savings rate chip:

Rate	Colour	Label
"e 20%	Green	Healthy

10–19%	Amber	Fair
< 10%	Red	Low

Savings rate = $\frac{(\text{income} - \text{totalMonthlyOutflow})}{\text{income}} \times 100$

Quick Actions

- **Add Subscription** — opens the subscription creation modal directly from the home screen
 - **Connect Bank** — deep-links to the Banks tab if no bank is connected
-

Behaviour Notes

- All amounts are shown in SAR.
 - Shared subscriptions display the user's **share cost**, not the full price.
 - VAT (15%) is added to any subscription where `includeVat`` is true before it is counted toward the total.
 - The dashboard updates reactively whenever any context (subscriptions, bills, loans) changes — no pull-to-refresh needed.
-

Feature: Subscriptions

Route: `/subscriptions` · File: `app/(tabs)/subscriptions.tsx`
Detail route: `/subscription/:id` · File: `app/subscription/[id].tsx`

Overview

The Subscriptions screen lists all recurring services the user pays for. It is the core feature of SubTrack. A connected bank account is required before the list is shown (the screen shows a "Connect a bank" gate otherwise).

Adding a Subscription

Tap the  FAB to open the creation modal. All fields:

Field	Type	Notes
Name	Text	Service name, e.g. "Netflix"
Category	Picker	Streaming, Software, Fitness, Food, Gaming, Utilities, Education, Other
Amount	Number	Base price in SAR
Billing Period	Picker	Weekly · Monthly · Annual
Start Date	Date (DD/MM/YYYY)	First charge date
Include VAT	Toggle	Adds 15% on top of the base amount
Shared	Toggle	Reveals member count stepper (1–6)
Trial	Toggle	Marks subscription as a free trial
Trial End Date	Date (DD/MM/YYYY)	Visible only when Trial is on

List & Filtering

- **Summary pill** — shows active subscription count and combined monthly spend at the top.
- **Status tabs** — All · Active · Cancelled
- **Category chips** — horizontal scroll of category filters; selecting one narrows the list.
- **Search** — full-text search across subscription names.

Duplicate Detection

SubTrack automatically scans for possible duplicate subscriptions (same service added more than once).

State	Behaviour
Banner	A yellow banner appears at the top: "Possible duplicates found — Review"
Review mode	Tapping "Review" filters the list to only the suspected duplicates
Highlight mode	A 4-second visual pulse highlights the duplicate cards in the list
Dismiss	The banner can be dismissed; it reappears if a new duplicate is added
Archive resolution	A duplicate can be resolved by archiving (cancelling) it directly from the review view

Subscription Detail Screen

Tap any subscription card to open its full detail view (`/subscription/:id``).

Hero Section

- Large service icon, name, merchant name, and status badges (category, shared, trial).

Amount Card (gradient)

- Period amount (e.g. SAR 34.99/month)
- ****Share amount**** — the user's personal cost when `isShared`` is true: ``amount / sharedMembers``
- Yearly equivalent: ``monthlyAmount × 12``
- Next billing countdown in days

Controls

Control	Description
Shared member stepper	1–6 people; updates share cost live
Include VAT switch	Adds 15% to displayed amounts
Trial switch	Toggles trial state
Trial end date input	DD/MM/YYYY; only visible when trial is on

Payment History

Up to 4 recent payment entries are shown, derived from the start date and billing cycle.

Actions

Action	Description
How to Cancel This	Opens AI advisor pre-loaded with a cancellation query for this specific service

Mark as Cancelled / Active	Toggles `status` between `active` and `cancelled`
Remove from SubTrack	Permanently deletes the subscription after confirmation

Shared Cost Splitting

When `isShared` is `true`:

- The user specifies how many people share the subscription (1 = just them, up to 6).
- `shareCost = amount / sharedMembers`
- The dashboard and analytics always use `shareCost` for this subscription, never the full amount.
- The AI Advisor also receives `shareCost` rather than the full price.

Trial Tracking

- Subscriptions can be marked as free trials with an end date.
 - A **Trial** badge appears on the card and detail screen.
 - The end date is displayed in a human-readable relative format (e.g. "Ends in 5 days").
-

Feature: Bills

Route: `/bills` · File: `app/(tabs)/bills.tsx`

The Bills screen tracks recurring household and utility bills that are not subscription-based.

Overview

Bills represent fixed recurring charges with a monthly due day — rent, electricity, phone, etc. Unlike subscriptions, they do not have a start date or billing cycle; they simply recur every month on the same day.

Adding a Bill

Field	Type	Notes
Name	Text	E.g. "DEWA Electricity"
Amount	Number	SAR
Due Day	Number (1–31)	Day of the month the bill is due
Category	Picker	Rent · Insurance · Utilities · Phone · Internet · Other

Bill List

- Each row shows the bill name, category icon, amount, and due day.
- A ****Paid**** toggle allows the user to mark a bill as paid for the current cycle.
- The toggle resets at the start of each calendar month.

Premium Gate

The Bills screen is gated behind SubTrack Premium. Non-premium users see an upgrade prompt instead of the list.

Feature: Loans

Route: `/loans` · File: `app/(tabs)/loans.tsx`

The Loans screen tracks credit card balances, BNPL instalments, and personal loans.

Loan Types

Type	Description
`credit_card`	Revolving credit with a limit and minimum payment
`bnpl`	Buy-now-pay-later (Tamara, Tabby, Spotii, custom)
`personal`	Personal or auto loan with fixed APR
`other`	Catch-all for non-standard debt

Adding a Loan

Field	Type	Notes
Name	Text	Descriptive label (e.g. "Tamara – iPhone 15")
Lender	Text	Optional
Type	Picker	credit_card · bnpl · personal · other
Current Balance	Number	SAR — remaining amount owed
Credit Limit	Number	SAR — for credit cards only
APR	Number	Annual Percentage Rate (%)
Minimum Payment	Number	SAR per month
Next Due Date	Date	ISO date
Installment Amount	Number	SAR — for BNPL
Total Installments	Number	BNPL total count
Paid Installments	Number	BNPL count paid so far

Payoff Simulator

Tap Simulate Payoff on any loan to open the payoff simulator.

- **Extra payment input** — the user enters an additional monthly payment amount.
 - The simulator calculates: new payoff date, total interest saved, and number of months saved.
 - Results update live as the extra payment amount changes.
-

Premium Gate

The Loans screen is gated behind SubTrack Premium.

Feature: Savings Goals

Route: `/goals` · File: `app/(tabs)/goals.tsx`

Overview

Savings Goals allow users to set financial targets (e.g. "Hajj fund", "New car down payment") and track progress toward them with manual contributions.

Adding a Goal

Field	Type	Notes
Title	Text	Goal name
Emoji	Picker	Visual icon (Ø=ÝŁ, Ø=Þ—, Ø<ßà, etc.)
Target Amount	Number	SAR
Deadline	Date (DD/MM/YYYY)	Optional; enables monthly rate calculation
Color	Color picker	Accent colour for the progress ring

Goal Cards

Each goal card shows:

- Emoji + title
- Progress ring (filled to `currentAmount / targetAmount`)
- Current amount / target amount
- Days remaining to deadline
- Monthly contribution rate needed

At-Risk Indicator

A goal is marked At Risk when the `monthlyRateNeeded` exceeds 20% of the user's income. The card shows a warning colour.

Contributing

Tap Add Savings on any goal card to record a contribution:

- Enter the amount in SAR.

- The contribution is appended to the goal's history log.
 - ``currentAmount`` is updated immediately.
-

Hijri Date Support

- Goal deadlines can be entered in Gregorian or Hijri format.
 - The detail screen displays both equivalents side-by-side via ``formatHijriDate()``.
-

Feature: Budgets

Route: `/budgets` · File: `app/budgets.tsx`

Overview

Budgets allow users to set monthly spending caps per subscription category. The budget screen shows current spend vs. the cap for each category.

Budget Health Indicators

Status	Condition	Colour
Normal	Spending < 80% of cap	Green
Near limit	Spending 80–100% of cap	Amber
Over limit	Spending > cap	Red

Adding / Editing a Budget

- Tap any category row to set or update its monthly cap.
- Caps are stored as `BudgetCap` records in AsyncStorage.
- Spend is calculated from all active subscriptions in that category (using `shareCost` where applicable).

Premium Gate

The Budgets screen is gated behind SubTrack Premium.

Feature: Bank Connections

Route: `/banks` · File: `app/(tabs)/banks.tsx`

Overview

The Banks screen allows users to connect Saudi bank accounts via the Lean Technologies open-banking API. Lean provides read-only access — no credentials are stored.

Supported Banks

Al Rajhi Bank · SNB · Riyad Bank · SABB · ANB · Alinma Bank · BSF · STC Pay

Connection Flow

1. User taps **Connect a Bank**.
 2. SubTrack creates a Lean customer via `POST /customers/v1/` using the stored `leanAppToken``.
 3. The Lean SDK presents a native consent screen (in-app browser).
 4. On success, SubTrack stores the bank account details locally and fetches the balance.
-

Auto-Detection

Once connected, SubTrack analyses recent transaction history to detect recurring charges:

- Known subscription providers are matched by merchant name.
 - Detected subscriptions are presented for the user to confirm and add to the tracker.
 - **Duplicate detection** flags any charge already tracked.
-

Income Verification

Lean can verify the user's monthly income from salary deposits. When verified:

- `AuthContext.income`` is updated.
 - `AuthContext.leanVerifiedAt`` is set to the ISO timestamp.
 - A "Verified" badge appears on the income field in Settings.
-

Premium Gate

Bank connectivity is a Premium-only feature.

Feature: Analytics

Route: `/analytics` · File: `app/(tabs)/analytics.tsx`

Overview

The Analytics screen provides visual spending insights across subscriptions, bills, and loans.

Charts & Metrics

Section	Description
Total Monthly Outflow	Combined spend: subscriptions + bills + loan minimums
Category Breakdown	Donut chart splitting spend by subscription category
Top Spending Category	Single highest-spend category with amount
Monthly Trend	6-month line chart showing total outflow per month
Income vs. Spending Ratio	Bar showing outflow as a percentage of user income
Savings Rate	$\frac{(\text{income} - \text{outflow})}{\text{income}} \times 100$; shown in % with colour coding
Subscription Creep Score	Trend indicator: are subscriptions growing month over month?
Year-over-Year Comparison	Current year total vs. same period last year

Savings Rate Display

- Requires income to be set (via Settings or Lean auto-detect).
- Green ($\geq 20\%$), Amber (10–19%), Red ($< 10\%$).
- Shown as both a percentage and an absolute SAR amount ($\text{income} - \text{outflow}$).

Feature: AI Advisor

Route: `/agent` · File: `app/agent.tsx`

Overview

The AI Advisor is a conversational financial assistant powered by OpenAI GPT-4o. It has full context about the user's subscriptions, bills, loans, and goals.

Context Sent to the Model

```
{
  "income": 18000,
  "monthlySubTotal": 650,
  "monthlyBillTotal": 2200,
  "monthlyLoanPayments": 1500,
  "totalDebt": 28000,
  "language": "en",
  "subscriptions": [
    { "name": "Netflix", "amount": 49.99, "shareCost": 24.99, "category": "Streaming" }
  ]
}
```

- Share cost is passed per subscription (not the full price).
- Language is forwarded so the model responds in the user's active language.

Quick Prompts

The initial screen shows four quick-action chips:

Chip	Pre-filled query
Spending overview	"Give me a breakdown of my monthly spending"
Savings tips	"How can I reduce my monthly outflow?"
Subscription audit	"Which subscriptions should I cancel?"
Debt strategy	"What's the fastest way to pay off my loans?"

Streaming

Responses are streamed via SSE (`POST /api/agent/chat`). The mobile app reads chunks and appends them to the message bubble in real time.

Cancellation Helper

From the Subscription Detail screen, tapping "How to Cancel This" opens the AI Advisor with a pre-loaded prompt: ``How do I cancel my [ServiceName] subscription?``.

Premium Gate

The AI Advisor is a Premium-only feature.

Feature: Premium

Route: `/premium` · File: `app/premium.tsx`

Free vs. Premium

Feature	Free	Premium
Subscription tracker (unlimited)	'	'
Bills tracker	—	'
Loan & BNPL payoff simulator	—	'
Savings goals	'	'
Analytics dashboard	'	'
Bank auto-detection (Lean API)	—	'
Duplicate charge detection	—	'
AI financial advisor	—	'
CSV/PDF export	—	'
Hijri calendar support	—	'
Advanced sharing	—	'
Budget caps	—	'

Pricing

Plan	Price
Monthly	SAR 19.99/month
Annual	SAR 149.99/year (37% saving)

Purchase Flow

1. User taps ****Upgrade to Premium**** from the premium screen or any premium gate.
2. Purchase is processed via Apple In-App Purchase or Google Play Billing.
3. On success, the server sets `isPremium = true` in `premium\_entitlements`.
4. App calls `GET /api/premium/:email` to confirm and updates `AuthContext`.

Premium Gate Component

`<PremiumGate>` wraps screens and components that require Premium. Non-premium users see an upgrade prompt with feature highlights instead of the gated content.

Feature: Settings

Route: `/settings` · File: `app/(tabs)/settings.tsx`

Profile Section

Setting	Description
Name	Display name (editable)
Email	Account email (read-only after registration)
Income	Monthly net income in SAR — used by Analytics and AI Advisor
Sign up / Log in / Log out	Auth actions

Appearance

Setting	Options
Theme	System · Light · Dark
Language	English · Arabic

Lean Token

For users with a custom Lean app token (developers / advanced users):

1. Tap **Lean Token** to open the token modal.
2. Paste the token from the Lean developer dashboard.
3. Tap **Save** — SubTrack calls `POST /customers/v1/` to create a Lean customer and stores the `customer\_id`.
4. The row shows a status badge: **Connected** (customer ID present) or **Token saved**.

Export

Tap Export Data to open the export preview modal.

Export Preview Modal

Section	Items counted
Subscriptions	Active + cancelled
Bills	All bills
Loans	All loans
Goals	All goals

Tapping Export CSV generates a comma-separated file containing all items and opens the native Share sheet.

Data Management

Clear All Data

Permanently deletes all subscriptions, bills, loans, budgets, and goals. A confirmation alert must be accepted. The app resets to its empty state immediately — no restart required.

About

Shows the app version number and build number.

API Reference

Base URL (development): ``https://<replit-dev-domain>/api-server/api``

Base URL (production): ``https://subtrack-budget.replit.app/api``

All authenticated endpoints require an ``Authorization: Bearer <jwt>`` header.

Authentication

``POST /api/auth/register``

Register a new user. Idempotent — re-registering with the same email returns success without overwriting existing data.

Request body

```
{
  "email": "user@example.com",
  "passwordHash": "sha256_hex_of_password",
  "name": "Ahmed Al-Rashid",
  "userId": "local-uuid-from-device"
}
```

Response ``201``

```
{ "ok": true }
```

Response ``400``

```
{ "error": "Email already registered" }
```

``POST /api/auth/login``

Authenticate and receive a JWT.

Request body

```
{
  "email": "user@example.com",
  "passwordHash": "sha256_hex_of_password"
}
```

Response ``200``

```
{ "token": "<jwt>" }
```

Response `401`

```
{ "error": "Invalid credentials" }
```

`GET /api/auth/profile`

Requires JWT

Retrieve the authenticated user's stored profile.

Response `200`

```
{
  "email": "user@example.com",
  "name": "Ahmed Al-Rashid",
  "userId": "local-uuid-from-device"
}
```

Premium

`GET /api/premium/:email`

Requires JWT (token's `sub` claim must match `:email`)

Check whether a user has an active premium entitlement.

Response `200`

```
{ "isPremium": true }
```

`POST /api/premium/:email`

Requires JWT · Development only (returns 403 in production)

Grant premium to a user.

Response `200`

```
{ "isPremium": true }
```

`DELETE /api/premium/:email`

Requires JWT · Development only (returns 403 in production)

Revoke premium from a user.

Response `200`

```
{ "isPremium": false }
```

AI Advisor

`POST /api/agent/chat`

Requires JWT

Send a chat message to the AI financial advisor. Returns a Server-Sent Events (SSE) stream.

Request body

```
{
  "messages": [
    { "role": "user", "content": "Which subscriptions should I cancel?" }
  ],
  "context": {
    "income": 18000,
    "monthlySubTotal": 650,
    "monthlyBillTotal": 2200,
    "monthlyLoanPayments": 1500,
    "totalDebt": 28000,
    "language": "en",
    "subscriptions": []
  }
}
```

Response — SSE stream:

```
data: {"content":"Here is my analysis..."}
data: {"done":true}
```

Notes

- The model used is `gpt-4o`.
- The system prompt enforces response language (`en` or `ar`) and SAR currency.
- Stream must be read with an SSE-aware client.

Health

`GET /api/healthz`

No authentication required.

Response `200`

```
{ "status": "ok" }
```

Error Codes

HTTP Status	Meaning
`400`	Bad request — missing or invalid fields
`401`	Missing, expired, or invalid JWT
`403`	Forbidden — action not permitted
`404`	Resource not found
`500`	Internal server error

JWT Details

Property	Value
Algorithm	HS256
Signing secret	`SESSION_SECRET` environment variable
TTL	90 days
Claim	`sub` = user email
Header format	`Authorization: Bearer <token>`

Localization

SubTrack ships with full bilingual support for English and Arabic, including complete RTL (right-to-left) layout switching.

Languages

Code	Language	Script direction
`en`	English	LTR
`ar`	Arabic	RTL

The active language is stored in `LanguageContext` and persisted in AsyncStorage. Switching language takes effect immediately across all screens — no restart required.

LanguageContext

```
interface LanguageContext {
  language: 'en' | 'ar';
  setLanguage: (lang: 'en' | 'ar') => void;
  t: (key: string) => string;    // Translation function
  isRTL: boolean;                // true when language === 'ar'
}
```

Usage in components

```
const { t, isRTL } = useLanguage();

<Text style={{ textAlign: isRTL ? 'right' : 'left' }}>
  {t('nav_dashboard')}
</Text>
```

RTL layout helpers

```
rtl.row()    // { flexDirection: isRTL ? 'row-reverse' : 'row' }
rtl.text()   // { textAlign: isRTL ? 'right' : 'left' }
```

Key Naming Convention

Keys use a ``screen_section_element`` prefix pattern:

Prefix	Module
<code>`nav_`</code>	Bottom tab labels
<code>`dash_`</code>	Dashboard screen
<code>`subs_`</code>	Subscriptions
<code>`banks_`</code>	Banks / Lean connection
<code>`bills_`</code>	Bills
<code>`loans_`</code>	Loans
<code>`goals_`</code>	Savings goals
<code>`analytics_`</code>	Analytics screen
<code>`agent_`</code>	AI Advisor
<code>`settings_`</code>	Settings
<code>`premium_`</code>	Premium screen
<code>`budget_`</code>	Budgets
<code>`common_`</code>	Shared UI strings (Save, Cancel, Delete, ...)

Dates & Numbers

Dates

- All date inputs use the **DD/MM/YYYY** format, consistent with the Saudi market convention.
- Goal deadlines display both **Gregorian** and **Hijri** equivalents via ``formatHijriDate()``.
- Loan payoff dates are localised: the month name is translated when sharing the payoff plan.

Numbers & Currency

- All monetary amounts are in **SAR (Saudi Riyal)**.
- Large numbers use ``toLocaleString()`` with the active locale for thousands separators.
- Arabic numerals (١ ٢ ٣ ٤ ٥ ٦ ٧ ٨ ٩) are not used — Western Arabic numerals are standard in Saudi financial interfaces.

Adding a New Key

1. Add the key and English value to ``locales/en.ts``.
2. Add the Arabic translation to ``locales/ar.ts``.
3. Use ``t('your_new_key')`` in the component.

If the Arabic translation is missing, ``t()`` returns the English fallback automatically.